The key steps involved in this process are

- *Store embeddings*—The first step is to save the generated embeddings in a database designed for vector storage. This can be a dedicated vector database (e.g., Pinecone, Milvus), which is purpose built for high-performance vector operations, or a general-purpose database that has added vector search capabilities (e.g., PostgreSQL with pgvector, MongoDB, Couchbase, Neo4j). This allows for efficient storage and retrieval of the embeddings.
- *Query processing*—When a user submits a query, we generate an embedding for that query using the same model that was used for the stored embeddings. This ensures that the query embedding is compatible with the stored embeddings, allowing for accurate comparisons.
- *Similarity search*—With the query embedding and the stored embeddings, we can employ various similarity search algorithms to identify the most relevant results. Popular algorithms include *cosine similarity*, which measures the cosine of the angle between two vectors (where a higher cosine value, closer to 1, indicates greater similarity), and *Euclidean distance*, which calculates the straight-line distance between two points (vectors) in a multidimensional space (where a shorter distance indicates greater similarity).
- *Return results*—Based on the similarity scores, we can retrieve and present the most similar items or answers to the user. This could be in the form of a ranked list of relevant information, such as articles, documents, or specific answers to the user's query.

Let's put all of those concepts into practice as we take a look at a very simple use case of an IT support knowledge base utilizing embeddings for the retrieval of similar Q/As or articles.

5.4.3 Use case: IT support knowledge base

Imagine a product designed to assist users with common IT-related questions and issues. By using embeddings and storing them in a database, the system can provide quick and relevant answers to user queries. Here's how it works:

1. *Embedding generation*—Each IT support question and its corresponding answer are converted into embeddings and stored in a database. This step will be performed before the application is deployed.
2. *User query*—When a user asks a question, the system generates an embedding for this query using the same model.
3. *Similarity search*—The system compares the query embedding with stored embeddings to find the closest match.
4. *Response*—The system retrieves and presents the corresponding answer to the user.

The following is a sample code that walks you through each step of the process using JavaScript.